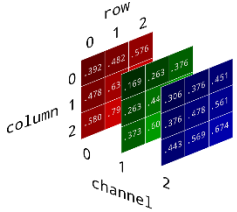
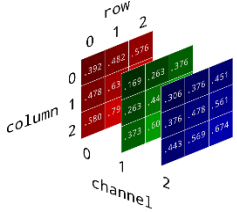
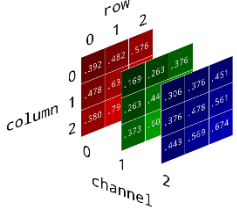
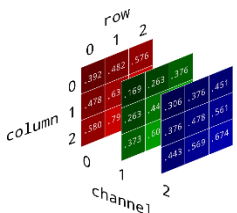


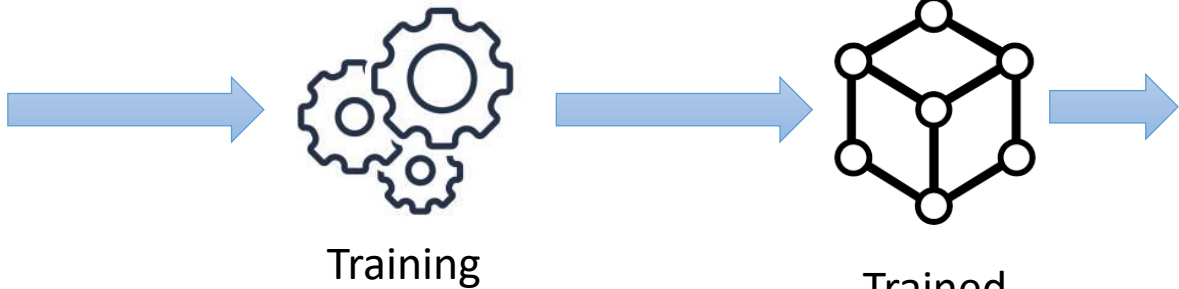
Introduction to Self-Supervised Learning

Daniel Moraes

Supervised Learning

Training data

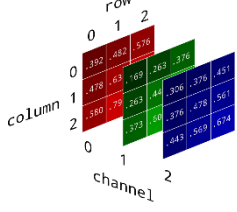
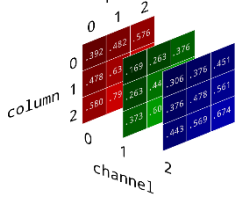
Data		Label
		Dog 0
		Cat 1
⋮	⋮	⋮
		Dog 0
		Bird 2

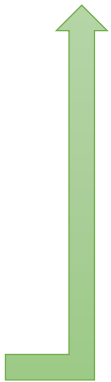


Predictions

0	Dog
2	Bird

New/Unseen Data

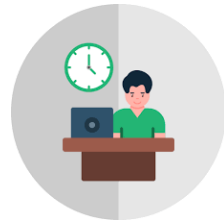
	
	



Supervised Learning

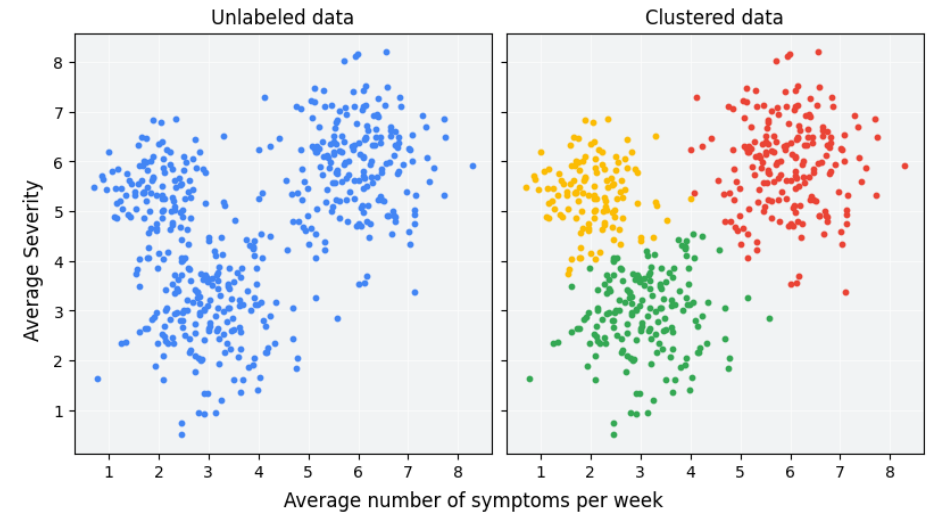
Training data

Data	Label	
	Dog	0
	Cat	1
⋮	⋮	⋮
	Dog	0
	Bird	2



- Expensive to acquire
- Time consuming
- May require domain knowledge (e.g. Land Cover)
- SoTA models are data hungry (e.g. Transformers)

Unsupervised Learning

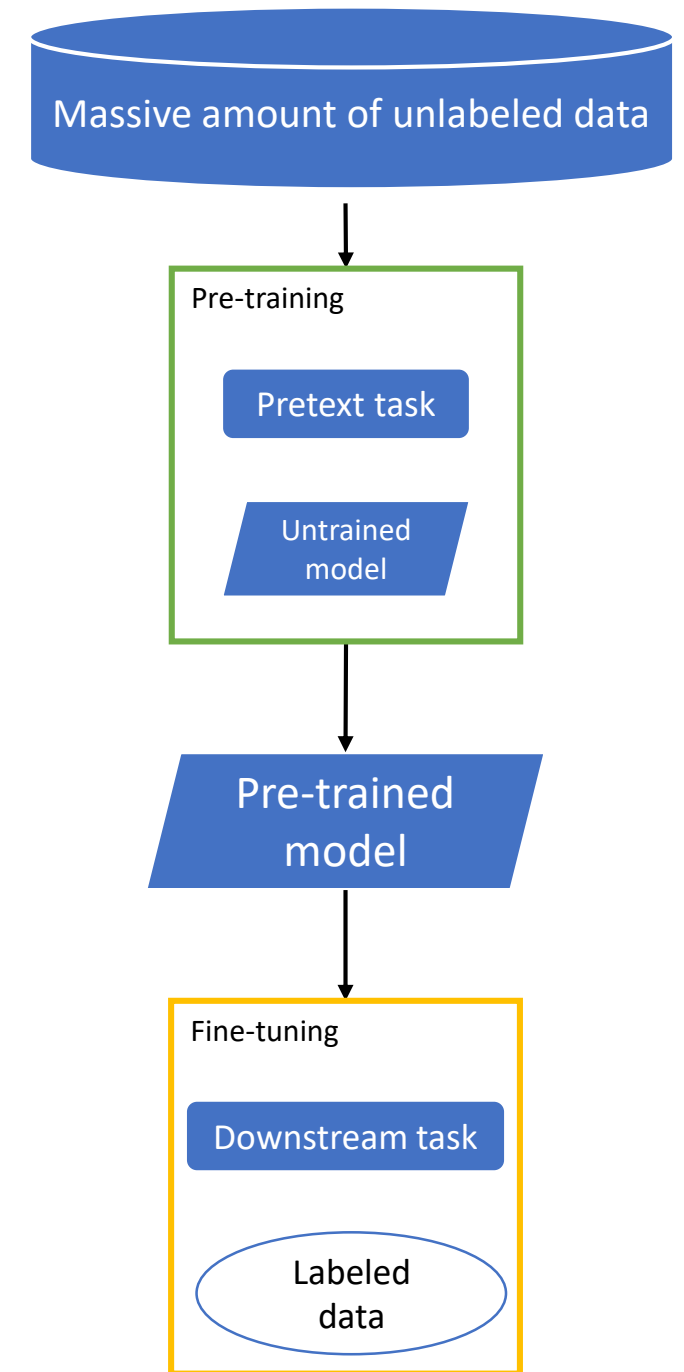


+ Does not require labeled data

- Suboptimal (worse performance)

Self-Supervised Learning

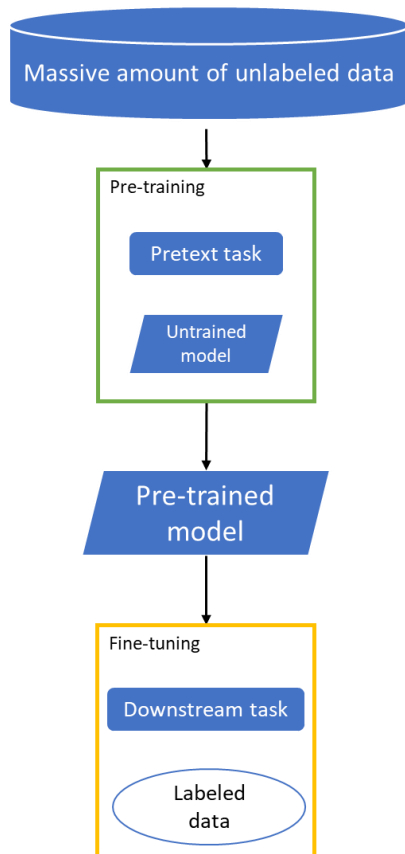
- Model learns from largely available unlabeled data
- Uses unlabeled data to create “labels”
- Key stages:
 - Pre-training: Learn useful representations training on a **pretext task**
 - Fine-tuning: use labeled data (supervised learning) to train the pre-trained model for a **downstream task**
- **Pretext tasks** (arbitrary task that helps learning representations)
 - Image Reconstruction (e.g. Masked Autoencoders)
 - Discriminate between positive and negative pairs (e.g. contrastive learning)
- **Downstream tasks** (typical end-user tasks)
 - Classification
 - Semantic Segmentation
 - Object detection



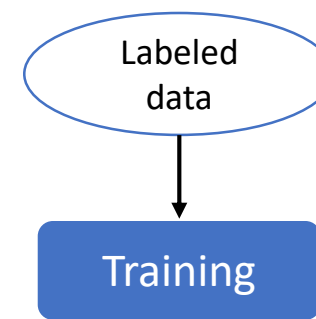
Self-Supervised Learning

Why use SSL?

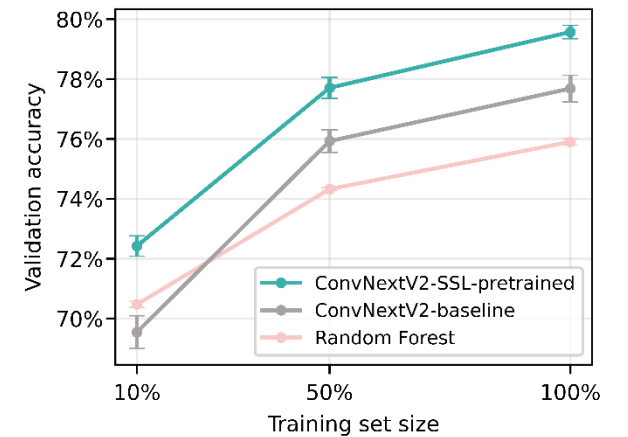
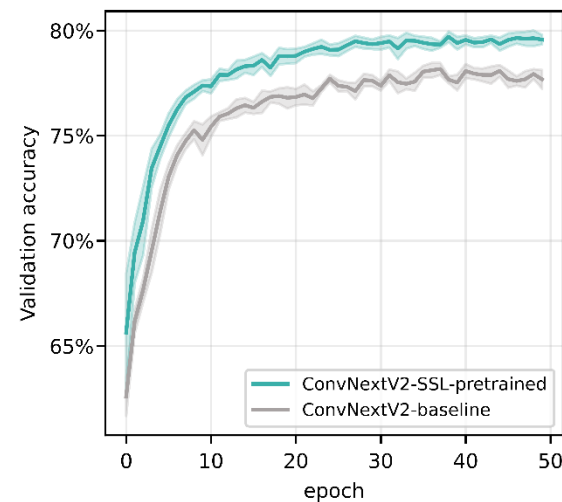
Fine-tune pre-trained model



Train model from scratch



>

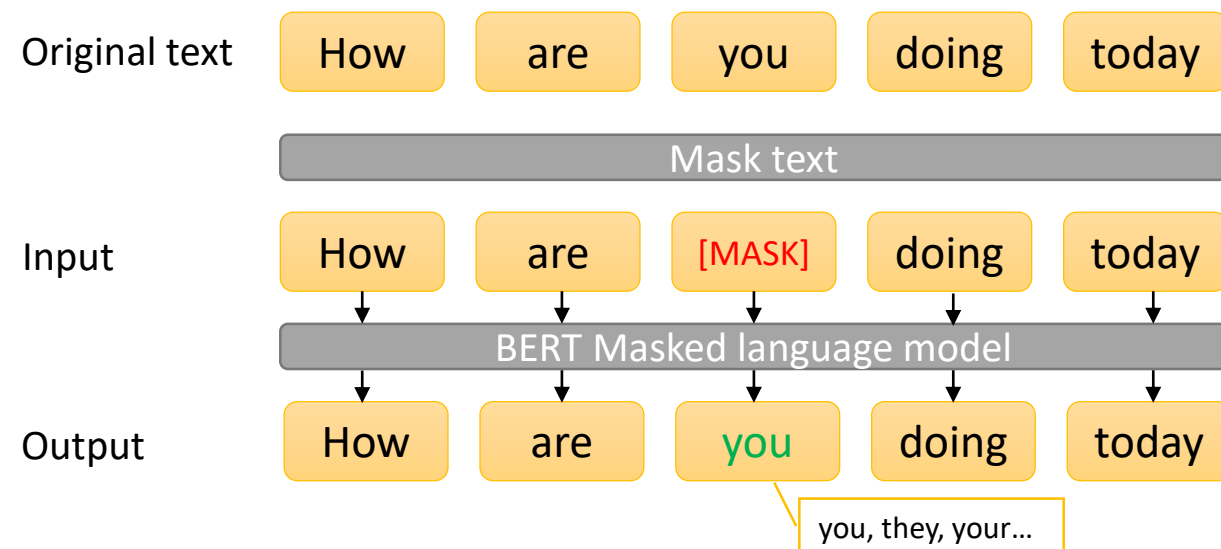


Self-Supervised Learning

“Uses unlabeled data to create ‘labels’”

“Learns **useful representations** training on a pretext task”

- The Natural Language Processing (NLP) example
 - Unlabeled data → Various text corpus available on the internet (blog posts, books, articles *etc*)
 - BERT (NLP model) → Pretext task = reconstruct original text (mask language modeling)



By reconstructing the original text, the model learns **useful representations**:

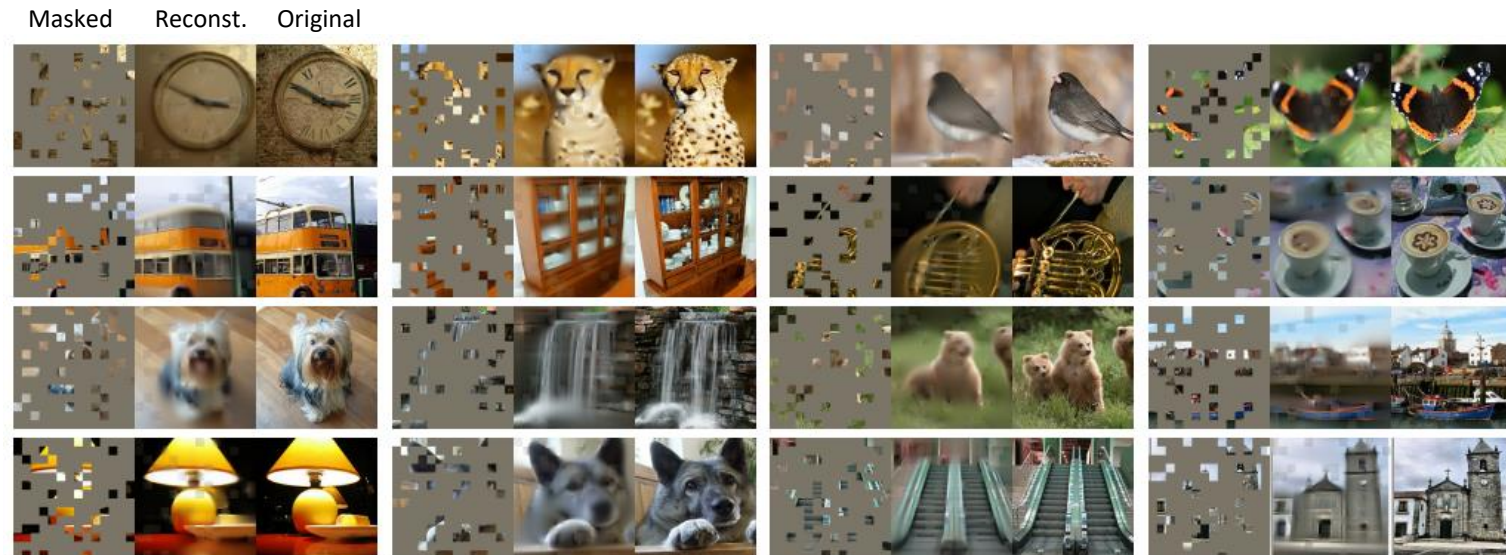
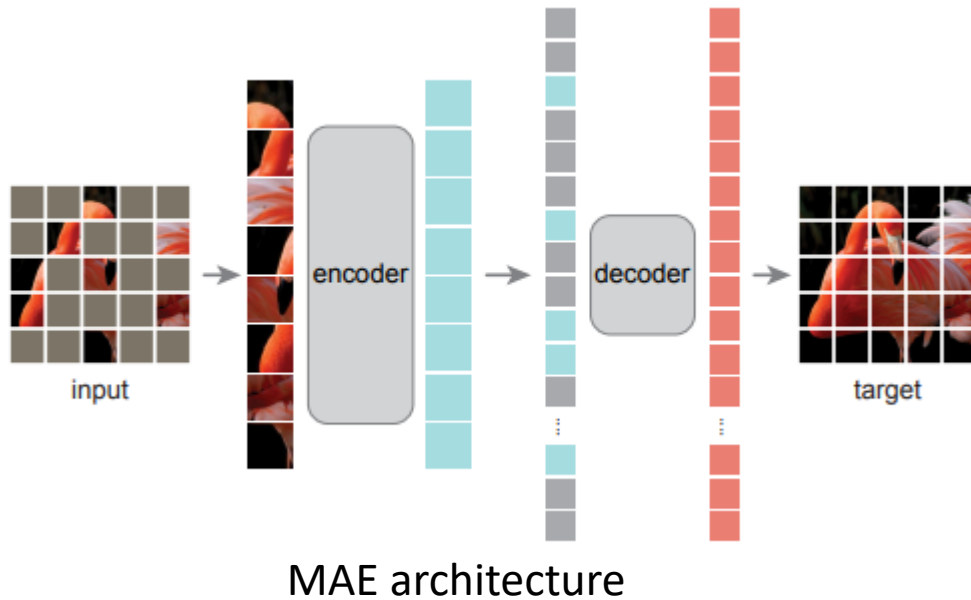
- Syntax and grammar (e.g. subject-verb agreement)
- Sentiment, intent, tone
- Semantics

Self-Supervised Learning in the Vision Domain

- Vast amount of unlabeled data
 - All internet imagery archive
 - Earth Observation data (Sentinel-2 generates ~1 TB of data every day)
- Most common pretext tasks
 - Image Reconstruction (Masked Image Modeling)
 - Masked Autoencoders
 - Discriminate between positive and negative pairs
 - Contrastive Learning

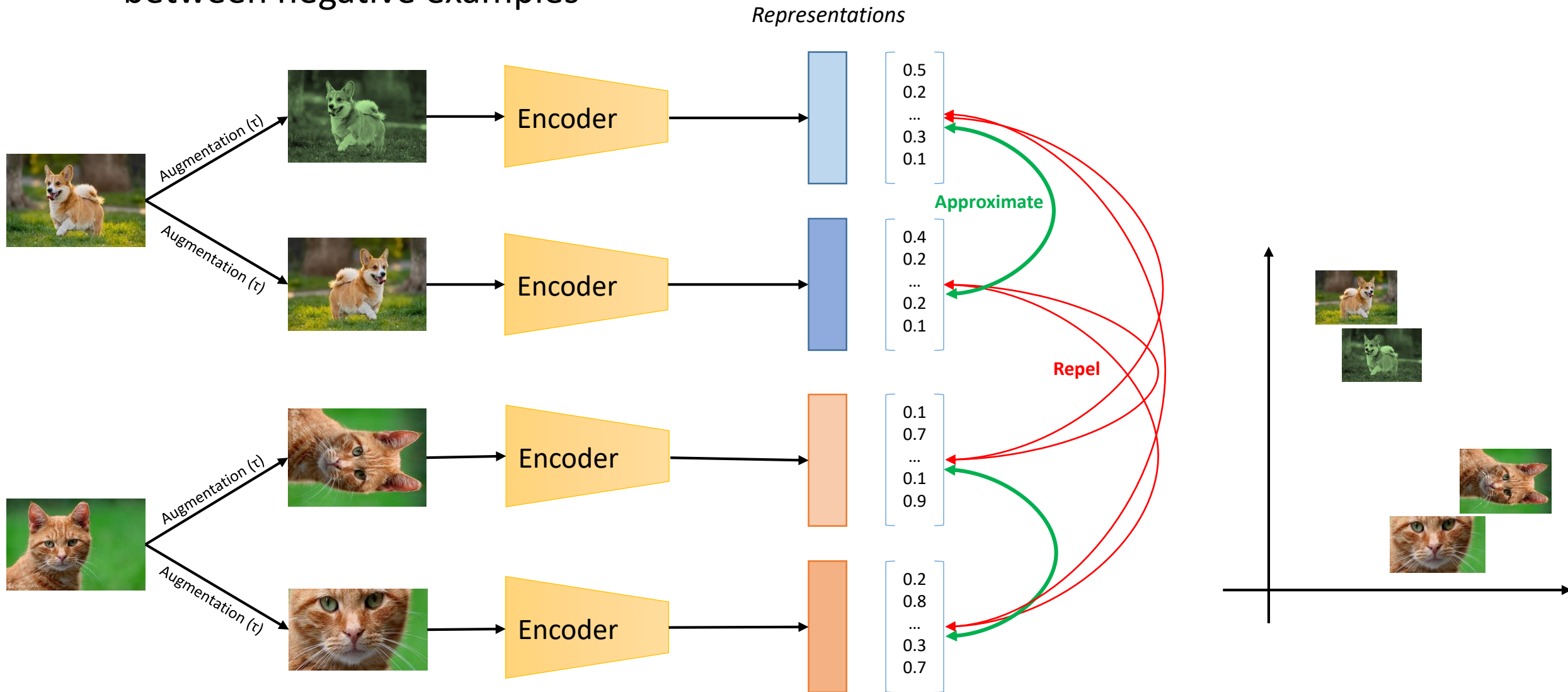
Masked Autoencoders (MAE)

- Pretext task: reconstruct original image after masking some patches



Contrastive Learning

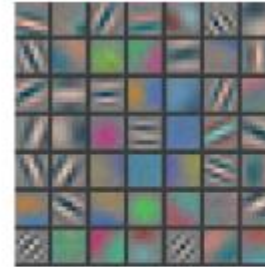
- Pretext task: maximize similarity between positive examples and minimize it between negative examples



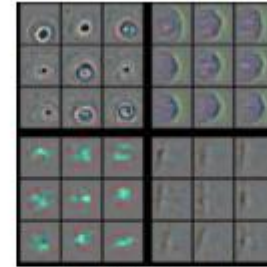
Representations in the Vision Domain

What are they?

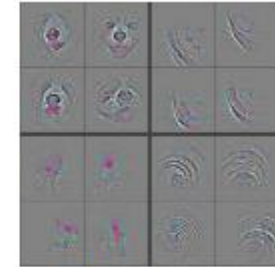
- Corners
- Edges
- Texture
- Shape
- Colors
- Object parts
- Unknown (???)



Low-level
features



Mid-level
features



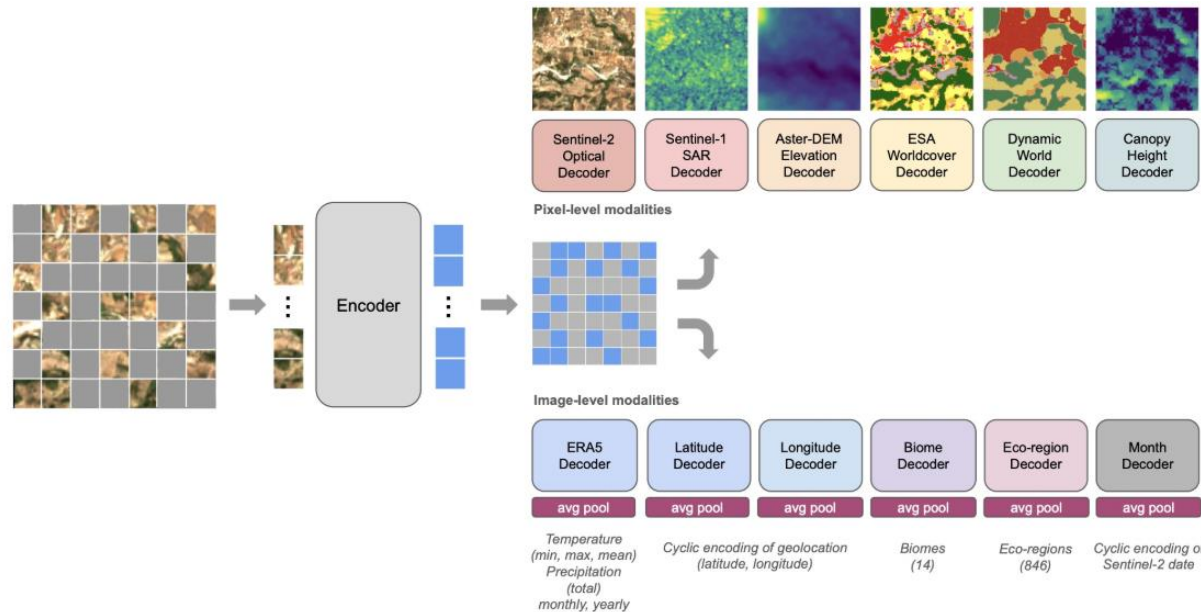
High-level
features



Object?

Geospatial Foundation Models (GFM)

- General-purpose models pre-trained on massive, diverse dataset
- Use Self-Supervised Learning for pre-training
 - Mostly MAE (e.g. SatMAE, Scale-MAE, MMEarth, Clay, etc)



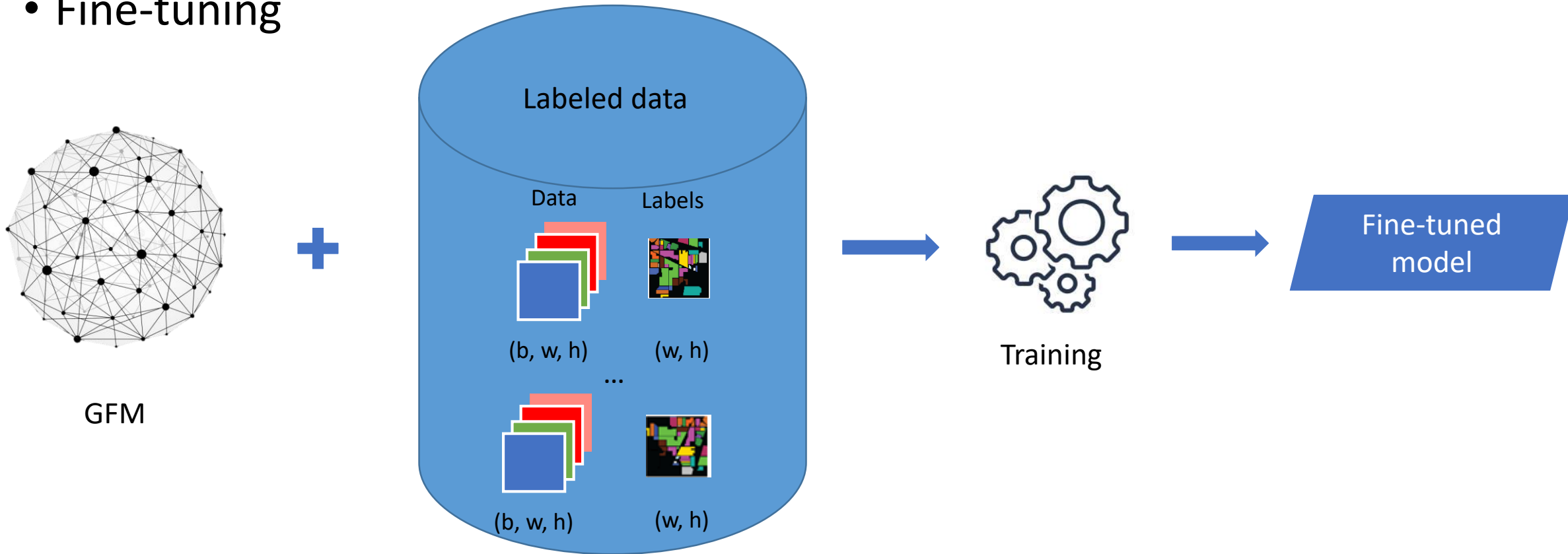
MMEarth GFM

Learned representations:

- Corners
- Edges
- Texture
- Objects (buildings, fields, roads)
- ???

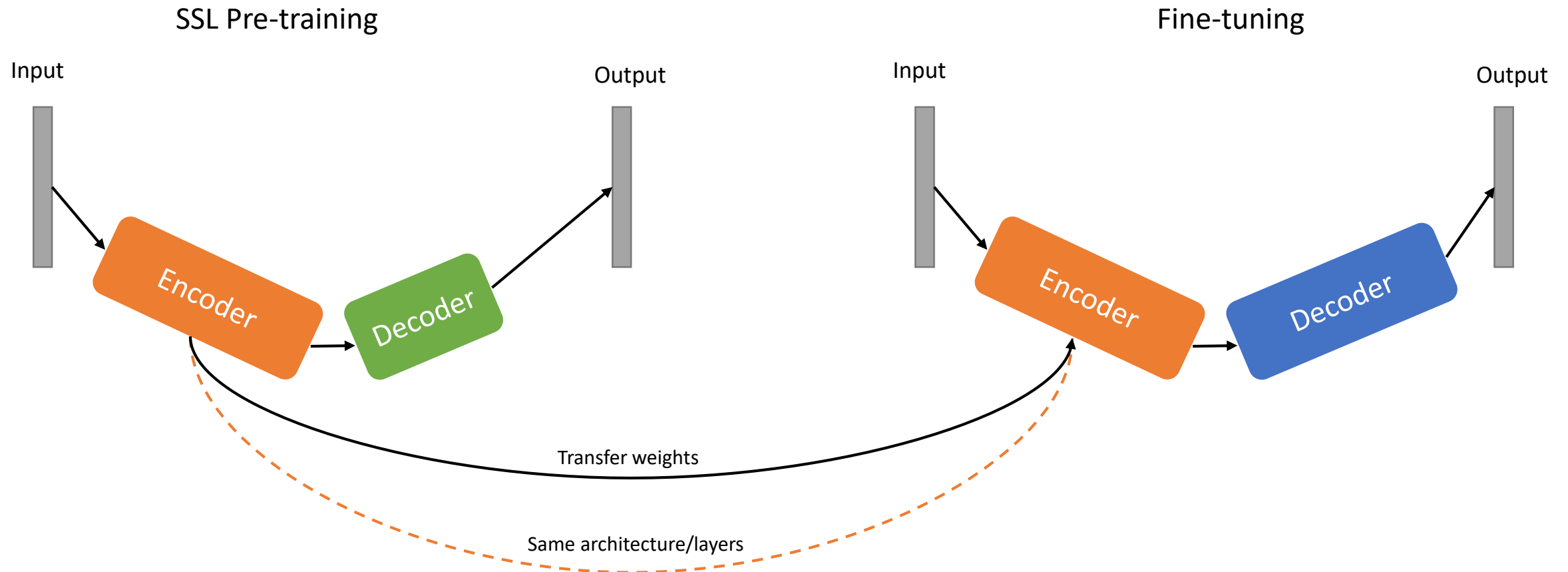
Geospatial Foundation Models (GFM)

- Fine-tuning



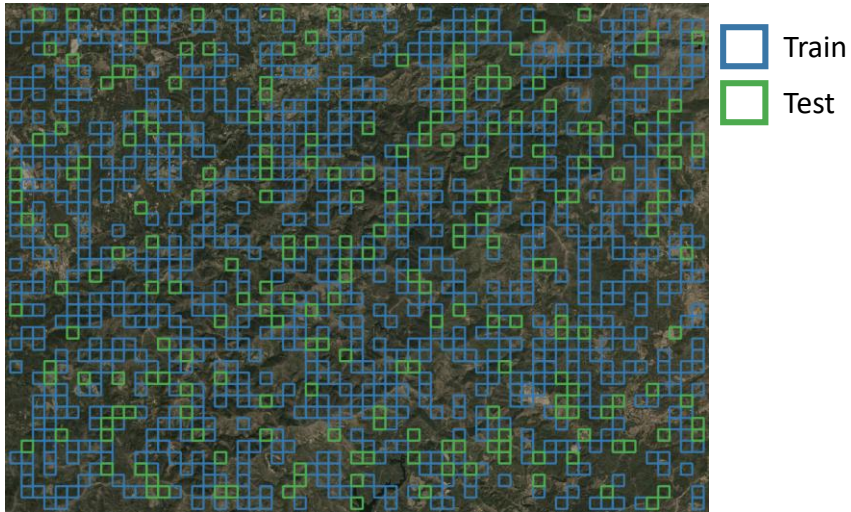
Model adaptation – Transfer Encoder only

- Representations are stored in the encoder, so the decoder normally does not matter
- We often want to plug in a different decoder



Example of workflow

1. Define image chips polygons (e.g. 64 x 64 pixels)

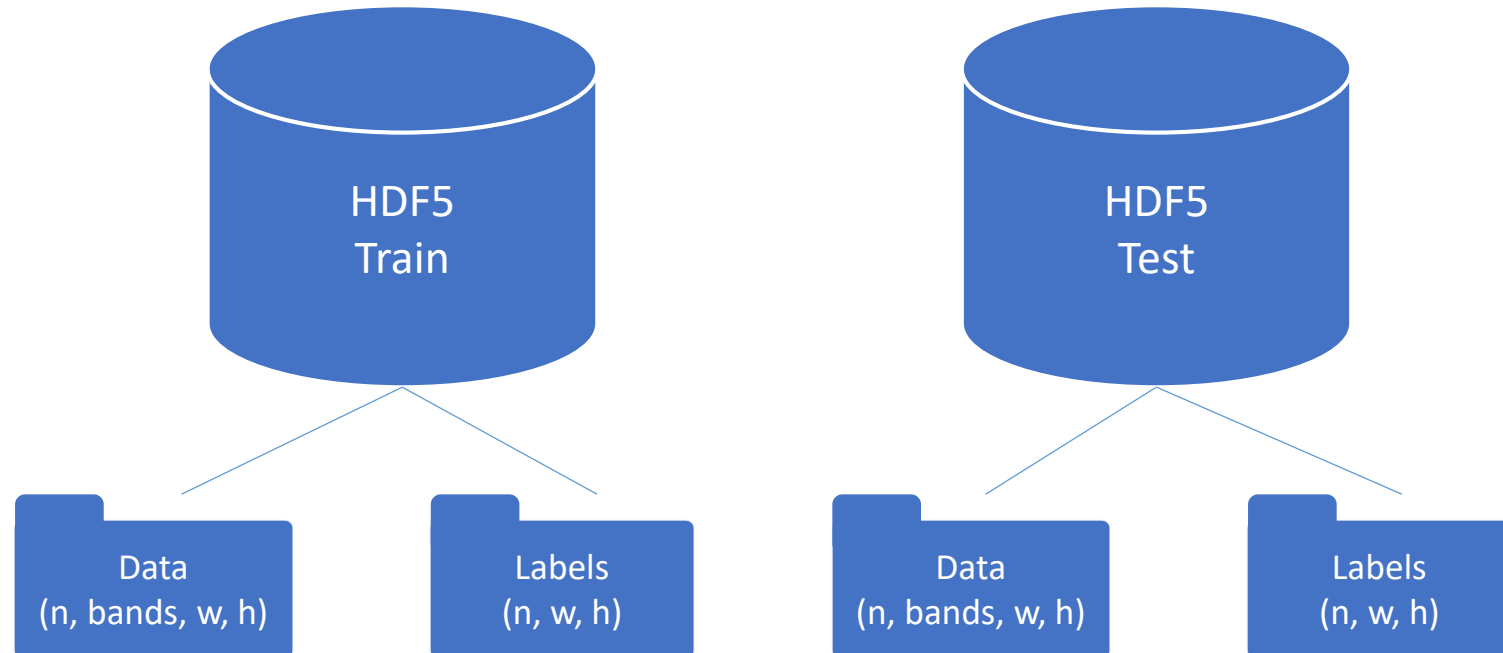


2. Crop original tif using polygons and create image chips

3. Crop labels tif using polygons and create labels chips

```
minx, miny, maxx, maxy = chip_geom.bounds
window = rasterio.windows.from_bounds(minx, miny, maxx, maxy, transform=transform)
with rasterio.open(image_path) as src:
    crop = src.read(window=window)
```

4. Save chips as HDF5 files



Example of workflow

5. Train model normally

```
model = your_gfm_model

for epoch in range(n_epochs):

    model.train()

    optimizer.zero_grad()

    outputs = model(inputs)
    loss = criterion(outputs, labels)

    loss.backward()
    optimizer.step()

    running_loss += loss.item()
```

6. Predict on full image

```
with torch.no_grad():
    output = model(chunk_tensor) # Forward pass through the model
    predicted = torch.argmax(output, dim=1).squeeze().cpu().numpy() # Get the predicted class for each pixel
```

Using GFM for S2Change Project

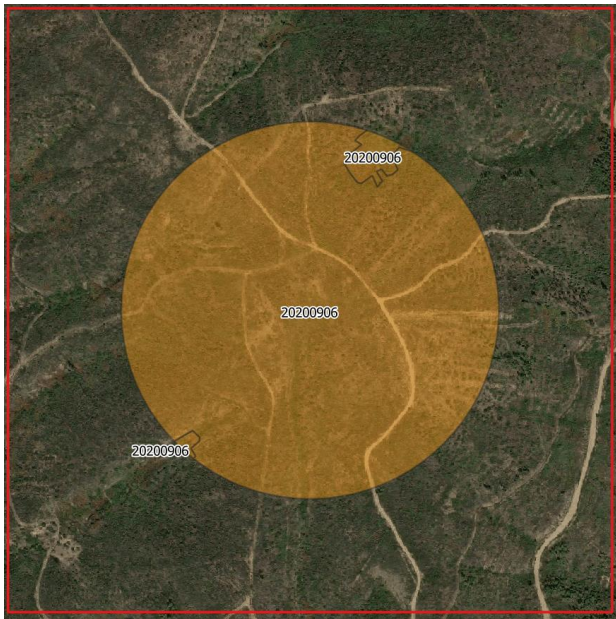
Aspects to be considered

- Which GFM?
- Trained on Sentinel-2/Landsat/similar?
- Shape of input data (e.g. 256 x 256, 64 x 64, etc)
- Input bands:
 - 10 Sentinel-2 bands?
 - 12 bands (6 pre change / 6 after change)?

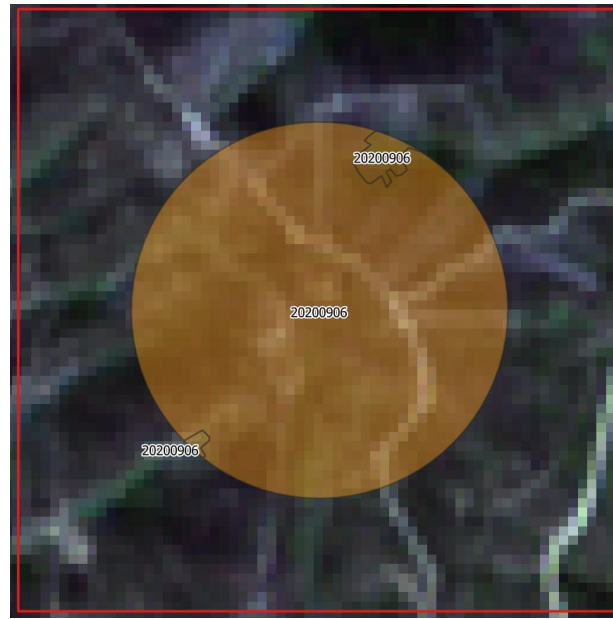
Framework for change detection refinement with DL

- Training

Create **64 x 64** pixel window around BDR300 buffers (ref. data)

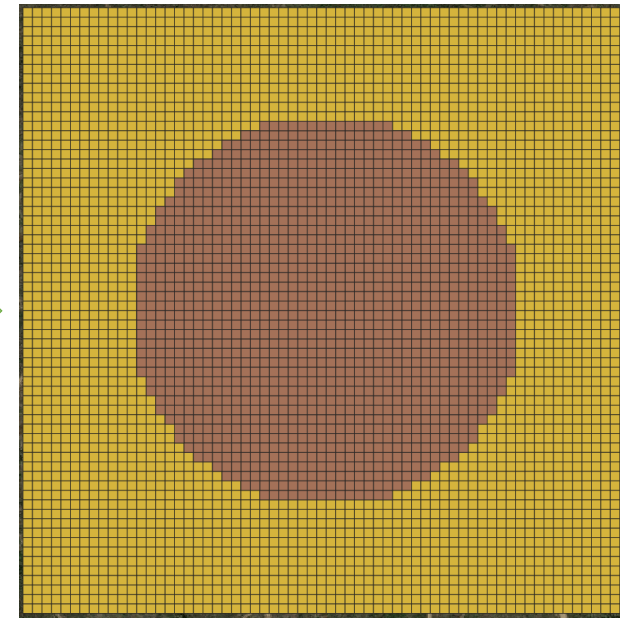


Create composite (12 bands: 6 bands pre change, 6 bands post change)



Create labels

Dummy value (outside mask)
Change



*use polygon's change date

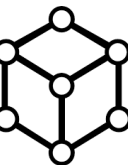
DATA



LABELS



Training



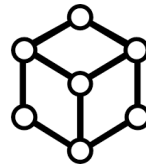
Trained Model

Change dates range: 2018-2021

Framework for change detection refinement with DL

- Prediction

Composite (6+6 bands)
Created every 2 months
Using pyccd breaks



Trained
Model



Map Change/No change

Framework for change detection refinement with DL

Aspects to be addressed

- How to fill the regions outside the mask?
 - Simply do not fill (leave with zeros)
 - Proposal: second composite with difference $\text{Months}_{3-4} - \text{Months}_{1-2}$

